

03 Matrix Multiplication



3.5 Block Matrix Multiplication

Summarizing the four views, and the general rules of block matrix multiplication

Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

Core Skills for This Section

1

Block matrix multiplication: matching sub-matrix shapes is the precondition for multiplying blocks

3

Extending the first view: cutting A into several row-blocks, B into several column-blocks

5

Extending the second view: cutting A into several column-blocks, B into several row-blocks

7

The fourth view of matrix multiplication: linear combination of row vectors — A cut into rows

2

The first view of matrix multiplication: inner product — A cut into rows, B cut into columns

4

The second view of matrix multiplication: outer product — A cut into columns, B cut into rows

6

The third view of matrix multiplication: linear combination of column vectors — B cut into columns

How This Section Unfolds

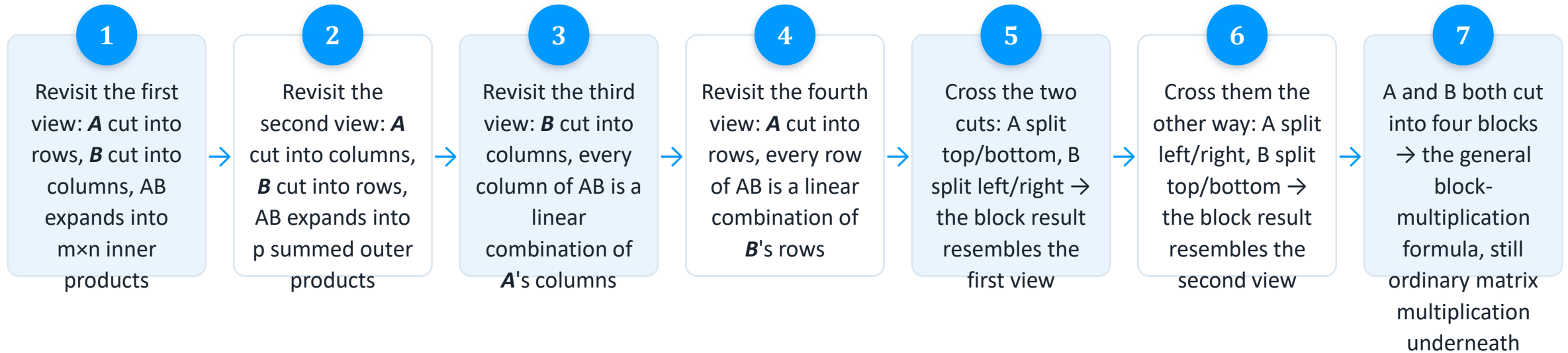


Table 1 · Comparing the Four Views of Matrix Multiplication

$AB = C$	1st View	2nd View	3rd View	4th View
Description	Inner product	Outer product	Linear combination of columns	Linear combination of rows
Partition	A into rows B into columns	A into columns B into rows	B into columns	A into rows
Expression	$c_{i,j} = \mathbf{a}^{(i)} @ \mathbf{b}_j$	$\mathbf{C}_k = \mathbf{a}_k @ \mathbf{b}^{(k)}$	$\mathbf{c}_j = \mathbf{A} \mathbf{b}_j$	$\mathbf{c}^{(i)} = \mathbf{a}^{(i)} \mathbf{B}$

Table 1 · Comparing the Four Views of Matrix Multiplication

$AB = C$	1st View	2nd View	3rd View	4th View
Meaning	Inner product of A 's row i and B 's column j	Outer product of A 's column k and B 's row k , summed	A 's columns weighted-combined by each column of B	B 's rows weighted-combined by each row of A
Shape	Determined by A 's row count $\times$ B 's column count	Each outer product is $m \times n$, same as the final matrix	A 's columns share dimension with C 's columns	B 's rows share dimension with C 's rows

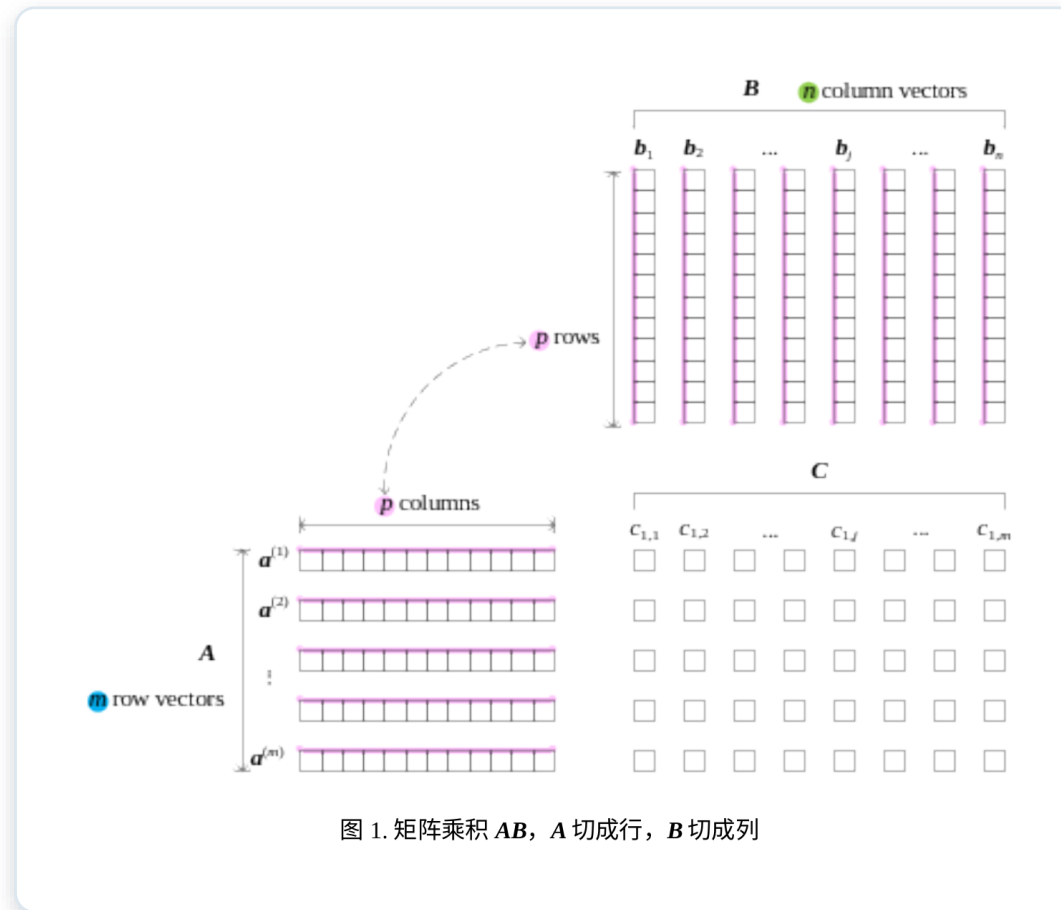
A Cut into Rows, B Cut into Columns

$$\mathbf{a}^{(i)} @ \mathbf{b}_j = c_{i,j}$$

- Cutting the left-hand matrix $\mathbf{A}$ into rows and the right-hand matrix $\mathbf{B}$ into columns writes $\mathbf{AB}$ as $m \times n$ separate matrix multiplications
- Each product $\mathbf{a}^{(i)} @ \mathbf{b}_j = c_{i,j}$ gives one entry of the result $\mathbf{C}$ — this is the first view of matrix multiplication, the inner product

Figure 1 · The Matrix Product AB , A Cut into Rows, B Cut into Columns

- A has m row vectors $\mathbf{a}^{(1)} \dots \mathbf{a}^{(m)}$, B has n column vectors $\mathbf{b}_1 \dots \mathbf{b}_n$; the shared dimension p “cancels out”
- Every entry $c_{i,j}$ of C is the inner product of A 's row i with B 's column j



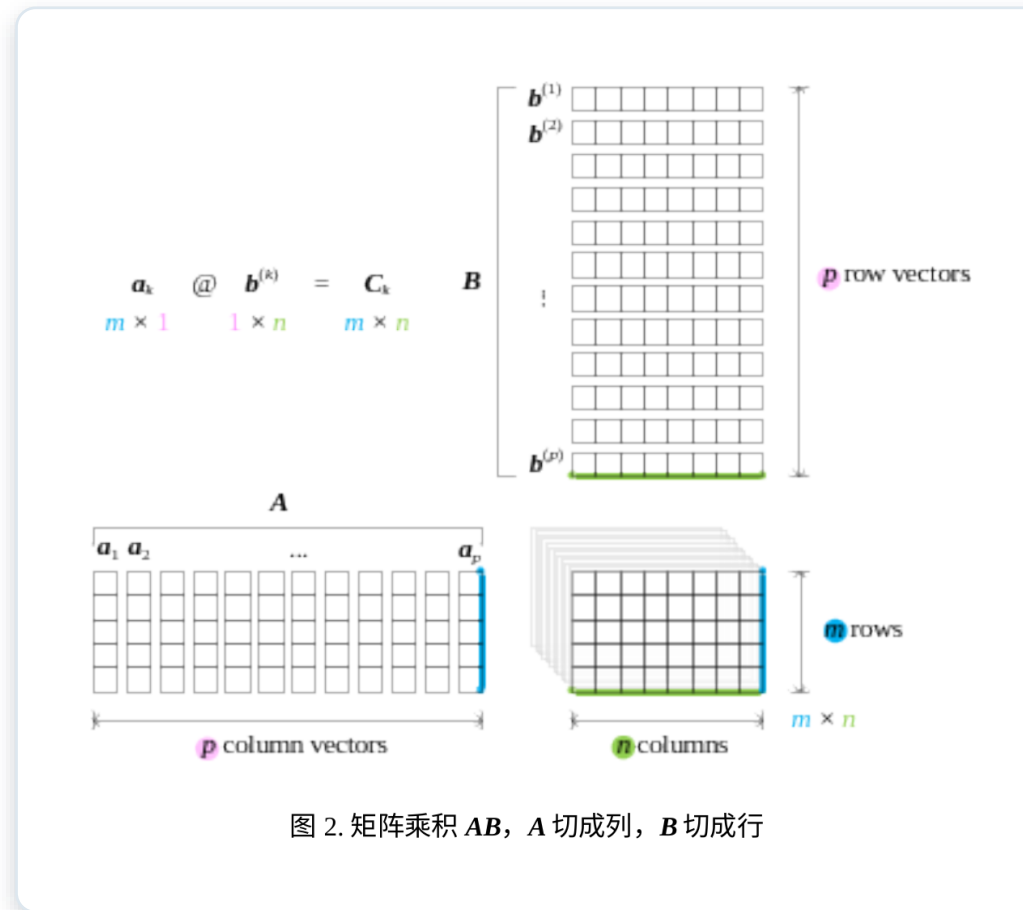
A Cut into Columns, B Cut into Rows

$$\mathbf{a}_k @ \mathbf{b}^{(k)} = \mathbf{C}_k$$

- Cutting the left-hand matrix $\mathbf{A}$ into columns and the right-hand matrix $\mathbf{B}$ into rows writes $\mathbf{AB}$ as a sum of p separate matrix products
- Each product $\mathbf{a}_k @ \mathbf{b}^{(k)} = \mathbf{C}_k$ is a matrix with the same shape as $\mathbf{C}$; summing the p matrices $\mathbf{C}_k$ gives $\mathbf{C}$ — this is the second view, the outer product

Figure 2 · The Matrix Product AB , A Cut into Columns, B Cut into Rows

- p is the number of columns in A and the number of rows in B — exactly the dimension that gets “canceled” in AB
- Each outer product $\mathbf{a}_k @ \mathbf{b}^{(k)}$ is an $m \times n$ matrix, the same shape as C ; summing all p of them entrywise gives C



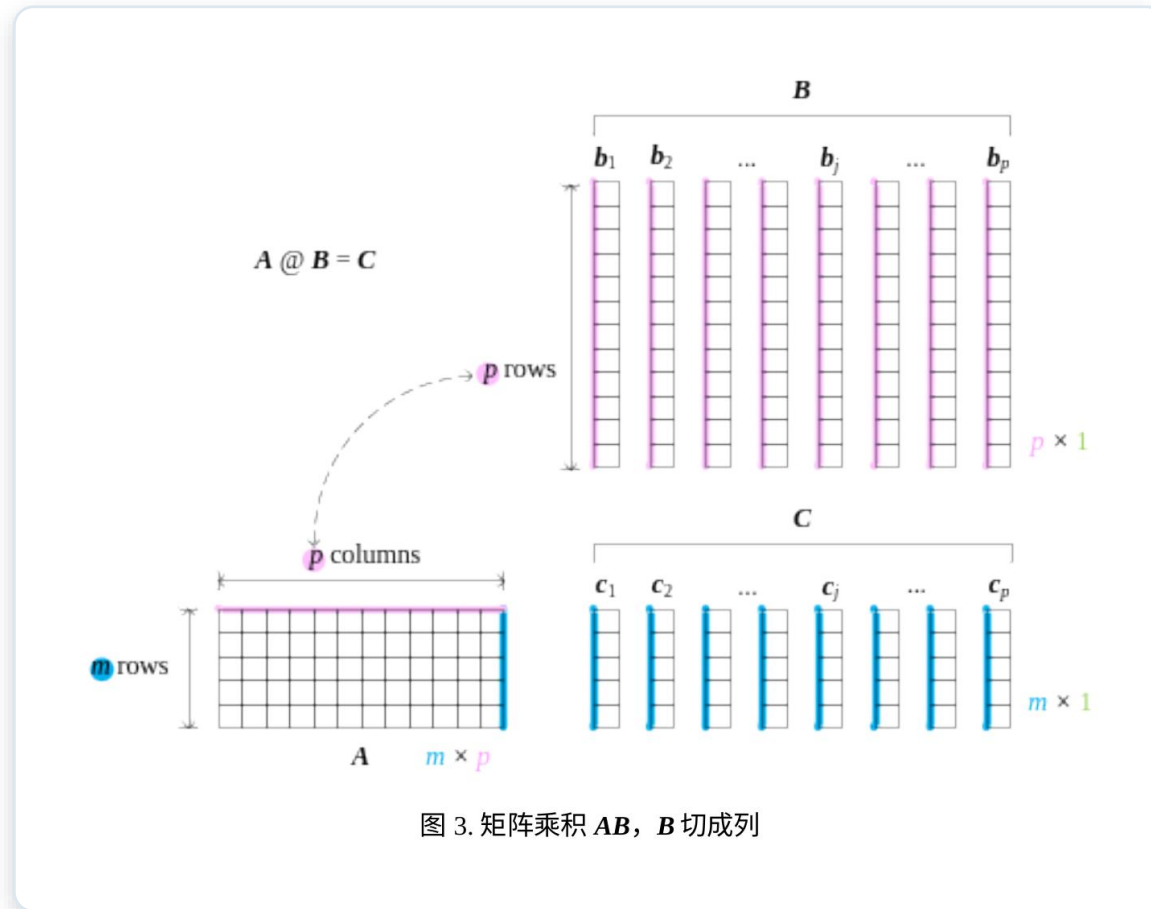
B Cut into Column Vectors

$$A[b_1 \ b_2 \ \cdots b_p] = [c_1 \ c_2 \ \cdots c_p]$$

- Cutting the right-hand matrix B into columns, every column of C becomes a linear combination of A 's columns: if $Ab_1 = c_1$, $Ab_2 = c_2 \dots Ab_p = c_p$, these p equations combine into the expression above
- This view deserves special attention — it reappears constantly in the projection operations later in this book

Figure 3 · The Matrix Product AB , B Cut into Columns

- A is $m \times p$, each column b_j of B is a $p \times 1$ vector; Ab_j gives c_j , an $m \times 1$ vector
- Looking at it in reverse, each column c_j of C can be assembled from A 's p columns, weighted by the entries of b_j



Similarly, Slice B Left and Right Once

$$\mathbf{AB} = \mathbf{A}[\mathbf{B}_1 \ \mathbf{B}_2] = [\mathbf{AB}_1 \ \mathbf{AB}_2]$$

- Slicing $\mathbf{B}$ left-right into two blocks $\mathbf{B}_1, \mathbf{B}_2$ expands $\mathbf{AB}$ into $[\mathbf{AB}_1 \ \mathbf{AB}_2]$ — each block is computed on its own and then placed side by side
- This is really the “coarse-grained” version of the linear-combination-of-columns view: bundling a group of columns into one block, the same rule still applies

Figure 4 · The Matrix Product AB , B Sliced Left and Right Once

- B_1 and B_2 are the left and right column-blocks of B ; each multiplied by A gives AB_1 and AB_2
- Placing AB_1 and AB_2 side by side reassembles the complete AB

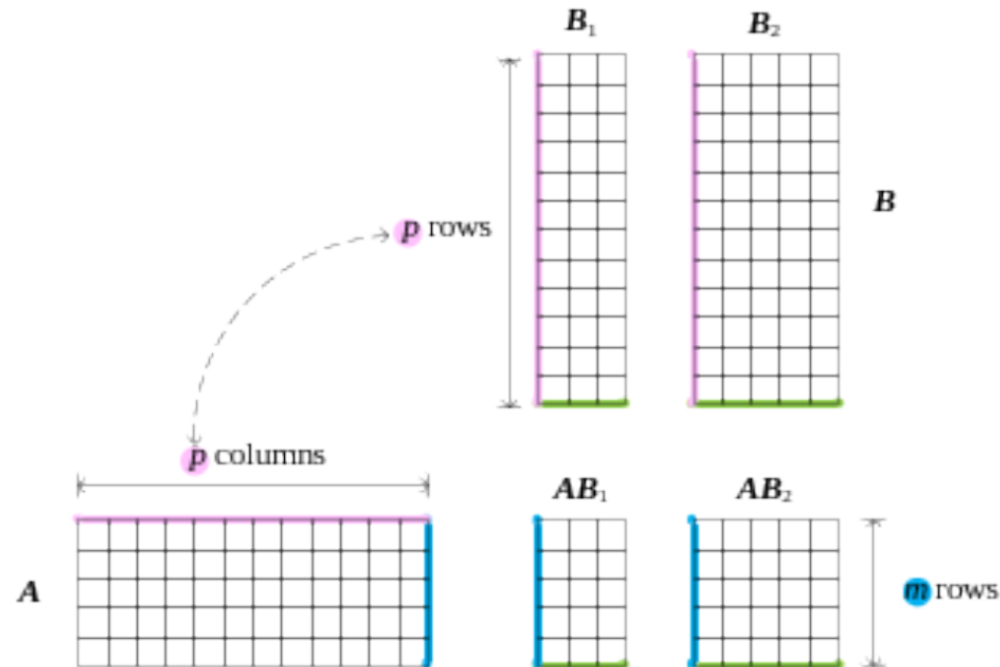
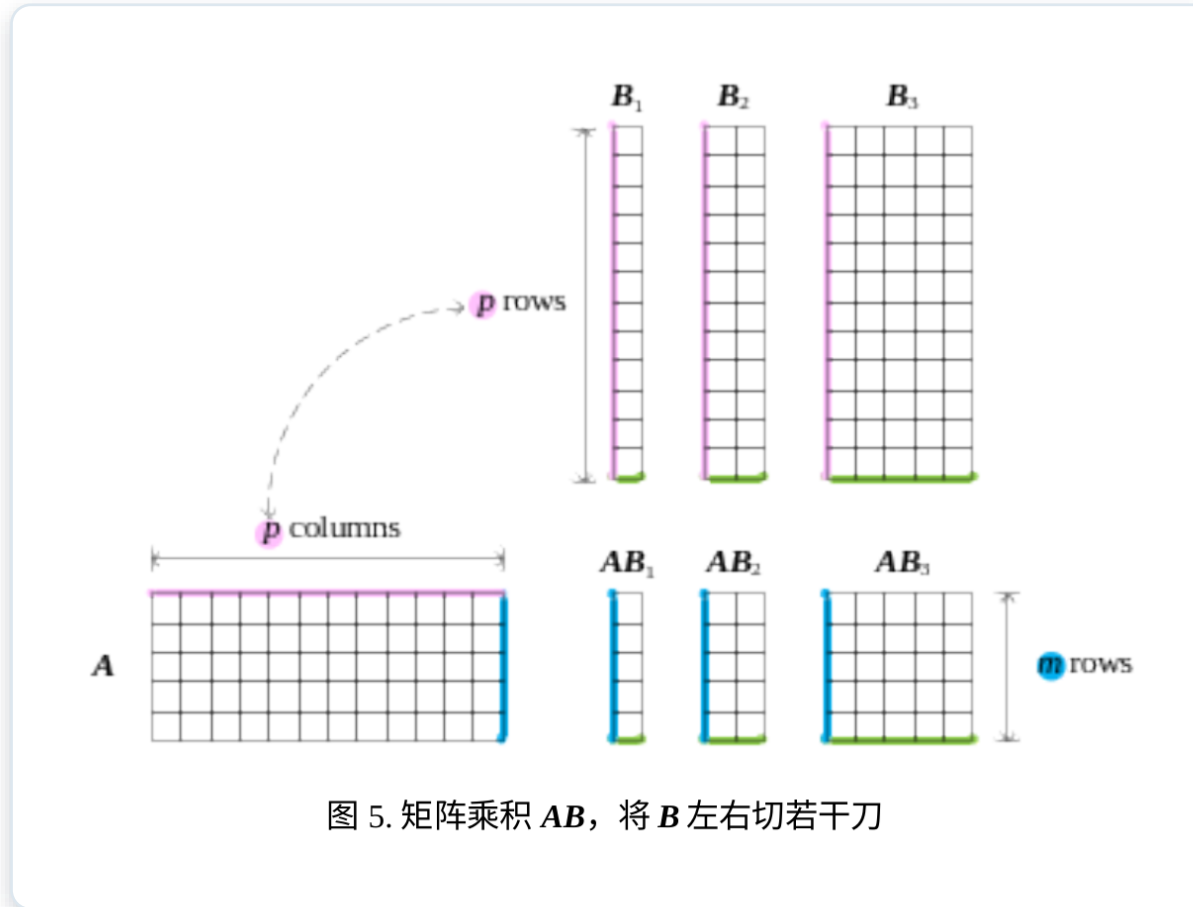


图 4. 矩阵乘积 AB ，将 B 左右切一刀

Figure 5 · The Matrix Product AB , B Sliced Left and Right Several Times

- Slicing B into any number of column-blocks $B_1, B_2, B_3 \dots$ the rule still holds: $AB = [AB_1 \ AB_2 \ AB_3 \ \dots]$
- No matter how many cuts, as long as each block's row count matches A 's column count, the block-wise result exactly equals the direct computation



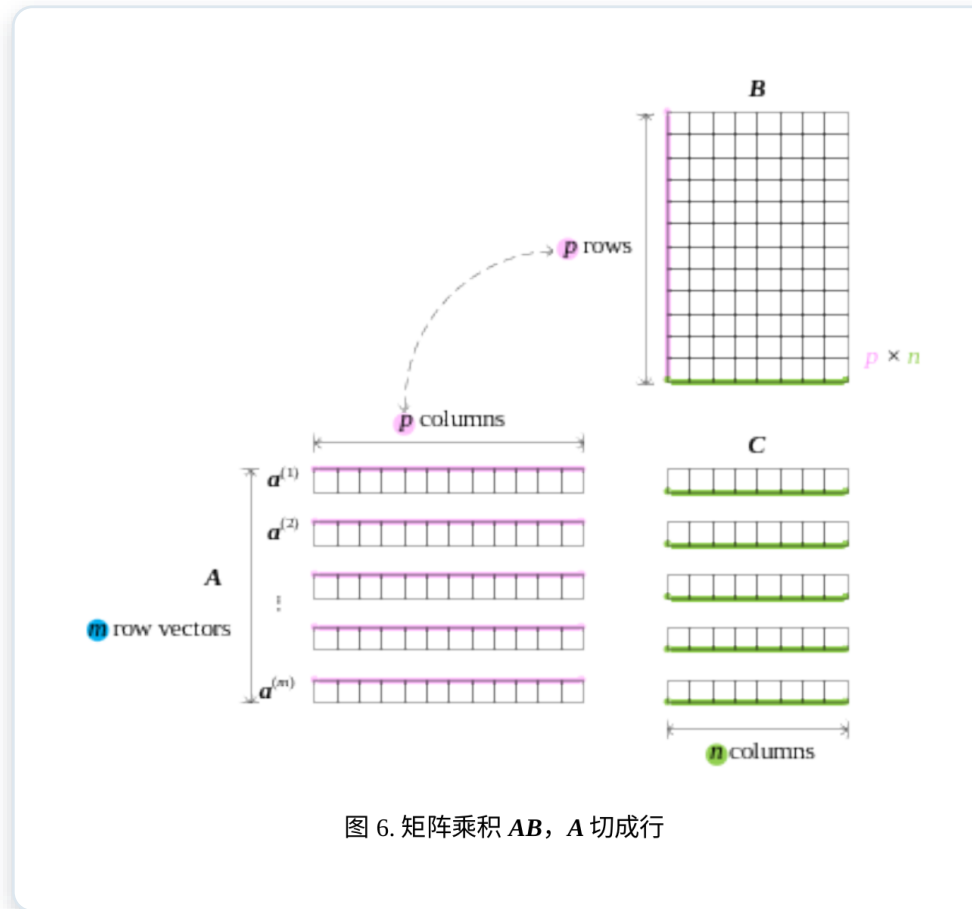
A Cut into a Set of Row Vectors

$$[\mathbf{a}^{(1)}; \mathbf{a}^{(2)}; \vdots; \mathbf{a}^{(m)}] \mathbf{B} = [\mathbf{a}^{(1)} \mathbf{B}; \mathbf{a}^{(2)} \mathbf{B}; \vdots; \mathbf{a}^{(m)} \mathbf{B}]$$

- Cutting the left-hand matrix $\mathbf{A}$ into rows, every row of $\mathbf{C}$ becomes a linear combination of $\mathbf{B}$'s rows
- Or read it the other way, as “assembly”: compute $\mathbf{a}^{(1)} \mathbf{B}$, $\mathbf{a}^{(2)} \mathbf{B}$... separately, then stack them in order to reassemble $\mathbf{C}$

Figure 6 · The Matrix Product AB , A Cut into Rows

- A has m row vectors $\mathbf{a}^{(1)} \dots \mathbf{a}^{(m)}$; each row vector multiplied by B on its own gives exactly the matching row of C
- This is the same relationship as Figure 1's $\mathbf{a}(i) @ B = \mathbf{c}(i)$ — here the emphasis is on the whole process of building C row by row



Slice A Top and Bottom Once

$$\mathbf{AB} = [\mathbf{A}_1; \mathbf{A}_2]\mathbf{B} = [\mathbf{A}_1\mathbf{B}; \mathbf{A}_2\mathbf{B}]$$

- Slicing $\mathbf{A}$ top-bottom into two blocks $\mathbf{A}_1, \mathbf{A}_2$ expands $\mathbf{AB}$ into $[\mathbf{A}_1\mathbf{B}; \mathbf{A}_2\mathbf{B}]$ — each block is computed on its own and then stacked
- This is the “coarse-grained” version of the linear-combination-of-rows view: bundling a group of rows into one block, the same rule still applies

Figure 7 · The Matrix Product AB , A Sliced Top and Bottom Once

- A_1 and A_2 are the top and bottom row-blocks of A ; each multiplied by B gives A_1B and A_2B
- Stacking A_1B and A_2B reassembles the complete AB

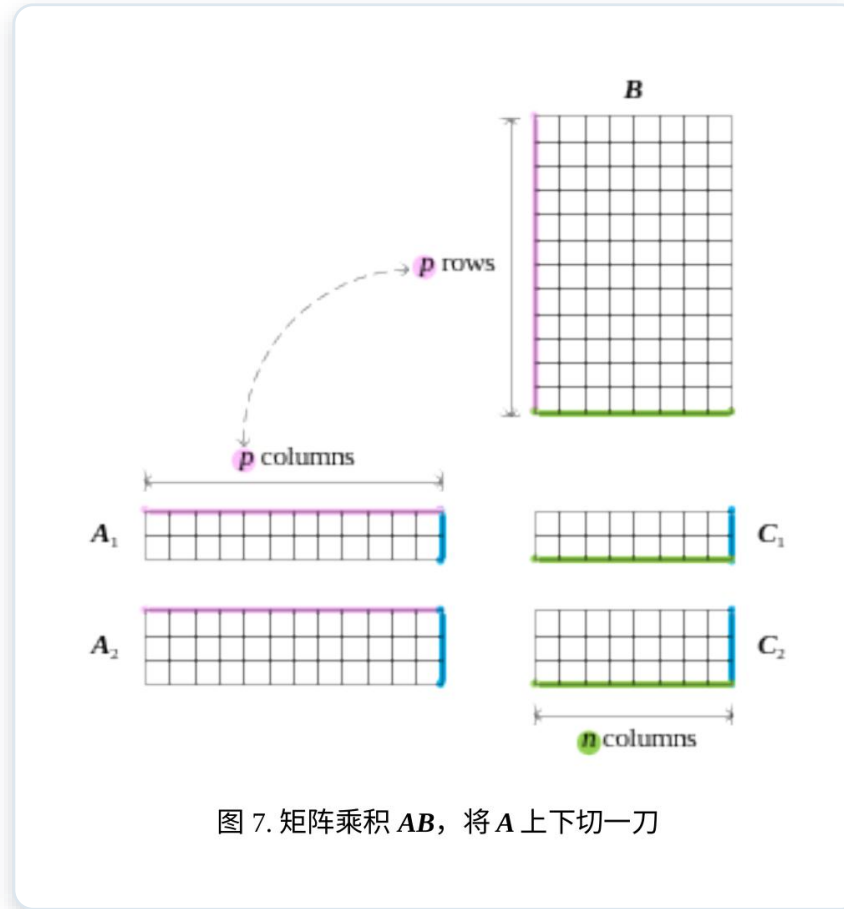
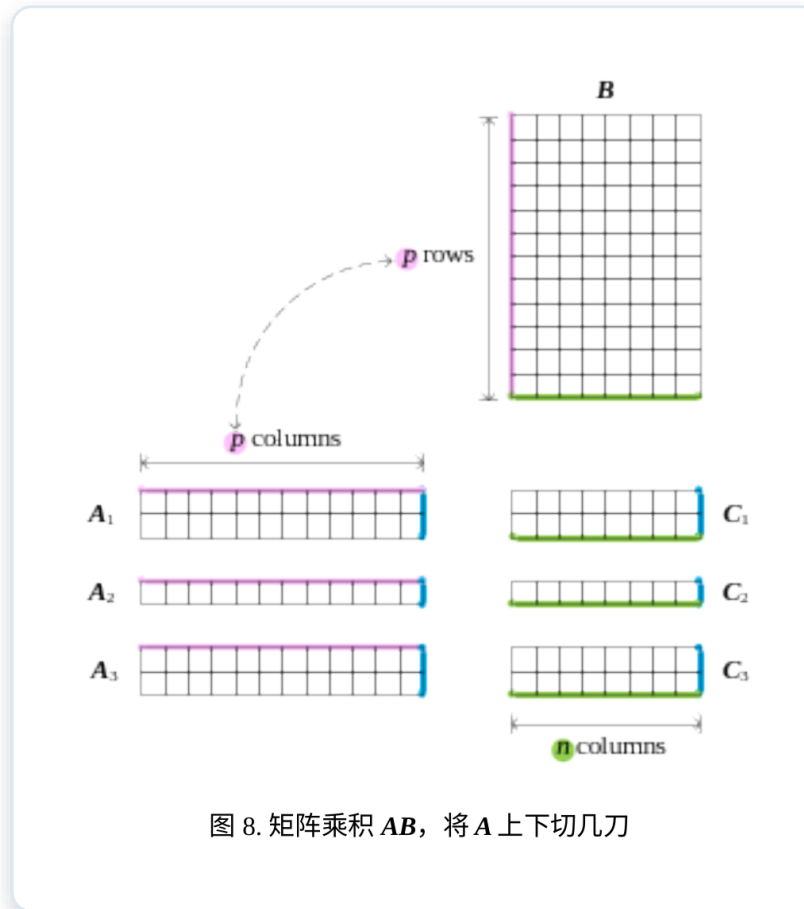


Figure 8 · The Matrix Product AB , A Sliced Top and Bottom Several Times

- Slicing A into any number of row-blocks $A_1, A_2, A_3 \dots$ the rule still holds: $AB = [A_1B; A_2B; A_3B; \dots]$
- No matter how many cuts, as long as each block's column count matches B 's row count, the block-wise result exactly equals the direct computation



A Split Top and Bottom, B Split Left and Right

$$\mathbf{AB} = [\mathbf{A}_1; \mathbf{A}_2][\mathbf{B}_1 \ \mathbf{B}_2] = [\mathbf{A}_1\mathbf{B}_1 \ \mathbf{A}_1\mathbf{B}_2; \mathbf{A}_2\mathbf{B}_1 \ \mathbf{A}_2\mathbf{B}_2]$$

- $\mathbf{A}$ split top-bottom times $\mathbf{B}$ split left-right expands into a 2×2 block matrix, each block a product of one sub-block of $\mathbf{A}$ with one sub-block of $\mathbf{B}$
- Think of $\mathbf{A}_1, \mathbf{A}_2$ as the two “entries” of $\mathbf{A}$, and $\mathbf{B}_1, \mathbf{B}_2$ as the two “entries” of $\mathbf{B}$ — this view resembles the first view of matrix multiplication

Figure 9 · The Matrix Product AB , A Split Top-Bottom, B Split Left-Right

- A_1 's and A_2 's column counts both equal B_1 's and B_2 's row counts — sub-matrix shapes must match for the blocks to multiply
- The result splits into four blocks $C_{1,1}$, $C_{1,2}$, $C_{2,1}$, $C_{2,2}$, each the product of one pair of sub-matrices

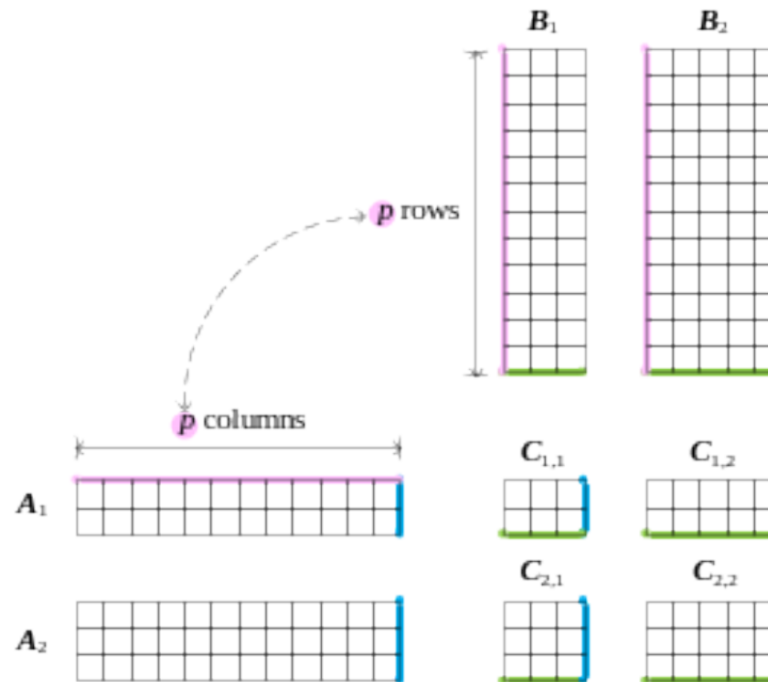


图 9. 矩阵乘积 AB , A 上下分块, B 左右分块

A Split Left and Right, B Split Top and Bottom

$$\mathbf{AB} = [\mathbf{A}_1 \ \mathbf{A}_2][\mathbf{B}_1; \mathbf{B}_2] = \mathbf{A}_1\mathbf{B}_1 + \mathbf{A}_2\mathbf{B}_2$$

- $\mathbf{A}$ split left-right times $\mathbf{B}$ split top-bottom expands into a sum of two sub-matrix products — $\mathbf{A}_1$'s column count equals $\mathbf{B}_1$'s row count, $\mathbf{A}_2$'s column count equals $\mathbf{B}_2$'s row count
- This resembles the second view discussed earlier: split the whole product into several block products, then sum them up block by block

Figure 10 · The Matrix Product AB , A Split Left-Right, B Split Top-Bottom

- A is sliced left-right into two column-blocks A_1 , A_2 ; B is sliced top-bottom into two row-blocks B_1 , B_2 , and the two are paired up
- A_1B_1 and A_2B_2 each share C 's shape; adding them entrywise gives C — exactly the same summed-outer-product pattern as the second view

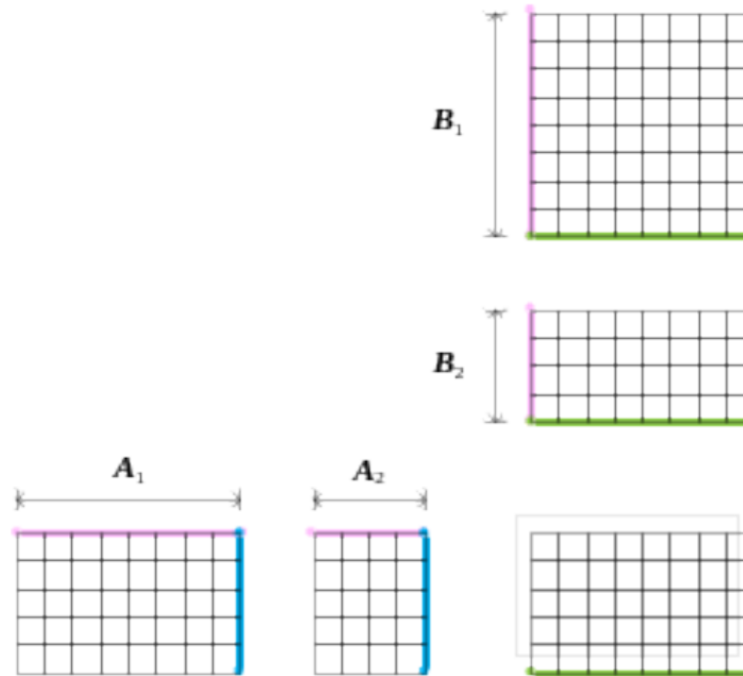


图 10. 矩阵乘积 AB , A 上下分块, B 左右分块

Figure 11 · The Matrix Product AB , A Split into Several Left-Right Blocks, B into Several Top-Bottom Blocks

- Slicing A and B even more finely, the rule still holds: $AB = A_1B_1 + A_2B_2 + A_3B_3 + \dots$
- No matter how many blocks, as long as each column-block of A matches the shape of the corresponding row-block of B , multiplying block by block and summing gives the full result

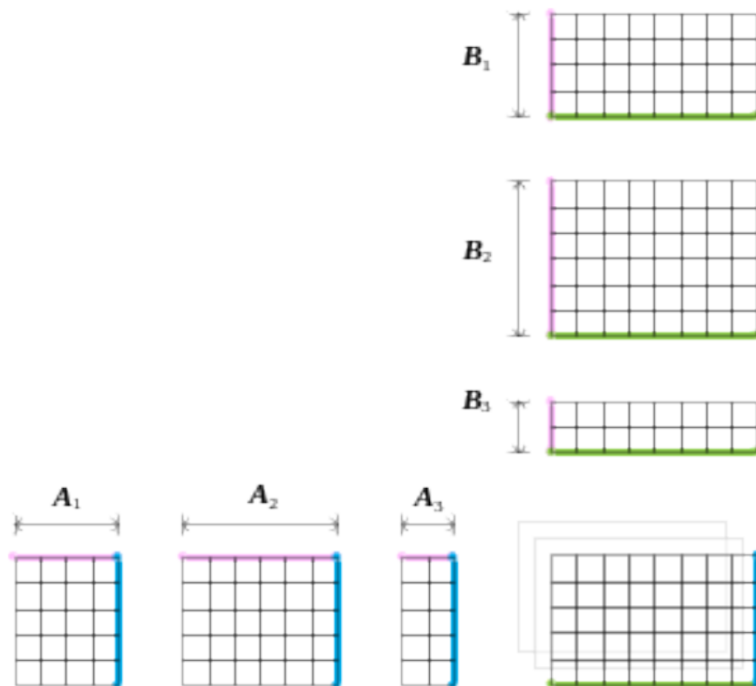


图 11. 矩阵乘积 AB , A 上下切几块, B 左右切几块

A and B Both “Cut into Four Pieces”

$$AB = \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{bmatrix} \begin{bmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{bmatrix} = \begin{bmatrix} A_{1,1}B_{1,1} + A_{1,2}B_{2,1} & A_{1,1}B_{1,2} + A_{1,2}B_{2,2} \\ A_{2,1}B_{1,1} + A_{2,2}B_{2,1} & A_{2,1}B_{1,2} + A_{2,2}B_{2,2} \end{bmatrix}$$

- A and B are both split top-bottom and left-right; the column counts of $A_{1,1}$, $A_{1,2}$, $A_{2,1}$, $A_{2,2}$ match the row counts of $B_{1,1}$, $B_{2,1}$, $B_{1,2}$, $B_{2,2}$ respectively
- This is equivalent to multiplying two 2×2 matrices, except each “entry” is now a sub-matrix, and the result C is likewise a 2×2 block matrix — this, too, is really the first view of matrix multiplication

Figure 12 · A and B Both Split Top-Bottom and Left-Right

- A is split into $2 \times 2 = 4$ blocks, and B likewise into 4 blocks; every block of C is a sum of products between A 's matching row-block and B 's matching column-block
- This closes the book's dedicated coverage of ... beginning: projection, least squares, eigenvalues, and more all build on these four

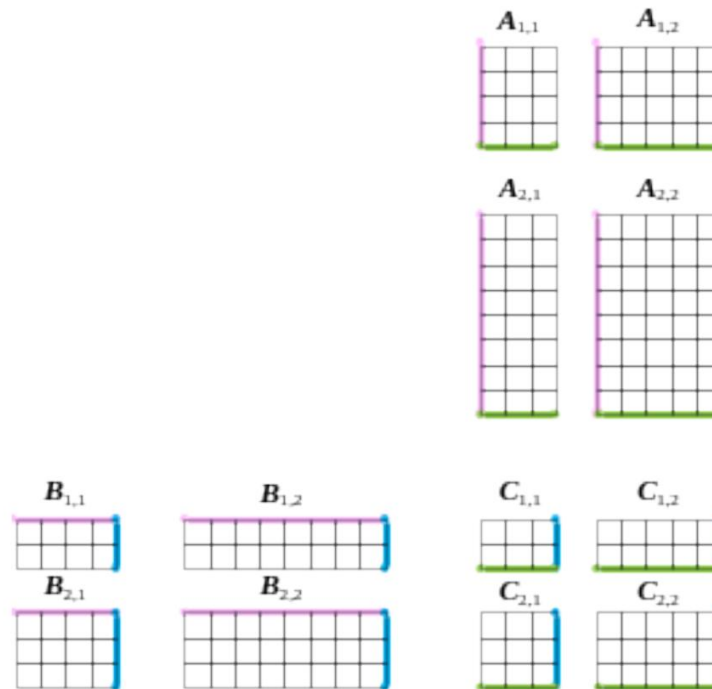


图 12. A 和 B 都上下左右分块

Tools and Applications for Block Matrix Multiplication

Slicing out sub-blocks In NumPy, $A[:m1, :]$ and $A[m1:, :]$ correspond to a top-bottom split, while $A[:, :p1]$ and $A[:, p1:]$ correspond to a left-right split — slicing itself adds almost no overhead

np.block() `np.block([[A11, A12], [A21, A22]])` stitches four sub-matrices directly into one complete matrix — a direct code expression of the block-partition idea

einsum, one unified notation `numpy.einsum` can describe the inner-product, outer-product, column-combination, and row-combination views all with a single string expression — this section's Q1 assigns exactly this as self-study

Parallel and distributed computing Large-scale matrix multiplication is often sliced into blocks, distributed across CPU cores, GPUs, or even separate compute nodes for independent computation and later aggregation — block matrix multiplication is the theoretical basis of parallel linear-algebra algorithms

Batch matmul in deep learning The batch matrix multiplication common in neural networks is essentially a group of small matrices “packed” along a new dimension, each block multiplied independently

Four Views, One Matrix Multiplication

The inner product, the outer product, linear combinations of columns, and linear combinations of rows are all, at heart, specific forms that block matrix multiplication takes under different ways of slicing

- However $\mathbf{A}$ and $\mathbf{B}$ are sliced, as long as the sub-matrix shapes match up, the block-wise computation always gives exactly the same result as computing the whole thing at once
- Understanding and computing matrix multiplication by breaking it into sub-blocks is a tool that runs through projection, least squares, eigendecomposition, and much more later in this book

Key Terms in This Section

Block Matrix *A matrix sliced into sub-matrices by rows, by columns, or by both at once*

1st View · Inner Product *A cut into rows, B cut into columns, $c_{i,j} = a(i) @ b_j$*

3rd View · Linear Combination of Columns *B cut into columns, $c_j = Ab_j$ — every column of C is a weighted sum of A's columns*

Block Outer-Product Pattern *A split left-right, B split top-bottom, $AB = A1B1 + A2B2$, resembling the second view*

np.block() *NumPy function that assembles several sub-matrices into one complete matrix by block layout*

Matching Sub-Matrix Shapes *For block multiplication, a left block's column count must equal the matching right block's row count*

2nd View · Outer Product *A cut into columns, B cut into rows, $C_k = a_k @ b(k)$, summed over p outer products*

4th View · Linear Combination of Rows *A cut into rows, $c(i) = a(i)B$ — every row of C is a weighted sum of B's rows*

Block Inner-Product Pattern *A split top-bottom, B split left-right, giving a 2x2 block result, resembling the first view*

einsum *numpy.einsum — uses the Einstein summation convention to express a wide range of matrix/tensor operations uniformly*

GO FURTHER

Companion Code and Further Reading

Further Reading · **numpy.einsum** documentation

<https://numpy.org/doc/2.2/reference/generated/numpy.einsum.html>

Further Reading · **numpy.block** documentation

Learn how to assemble a complete block matrix from its sub-matrices

Review · the **LA_03_01 – LA_03_04** notebook series

Complete code for the four views of matrix multiplication — a great companion to Table 1 in this section

Open-Source Repository

Code and figures for the whole book: <https://github.com/Visualize-ML>

Questions Q1 – Q9

Q1 Study the Einstein Summation Convention on your own; see <https://numpy.org/doc/2.2/reference/generated/numpy.einsum.html>.

Q3 Slice $B=[1,4,7,10;2,5,8,11;3,6,9,12]$ left-right into two column-blocks B_1, B_2 (2 columns each); compute AB_1 and AB_2 in Python and concatenate them, then compare against $A @ B$.

Q5 Write a function `block_matmul_2x2(A1, A2, B1, B2)` implementing $AB=[A1;A2][B1\ B2]$, returning the 2×2 block result.

Q7 Use `np.block()` to assemble hand-computed sub-blocks $C_{1,1}, C_{1,2}, C_{2,1}, C_{2,2}$ into the full matrix C , and verify it equals $A @ B$.

Q9 Write a function `check_block_shapes(A1, A2, B1, B2)` that checks whether the four sub-matrices involved in a block multiplication have pairwise matching shapes, raising a `ValueError` if not.

Q2 Given $A=[1,2,3;4,5,6]$, $B=[1,4;2,5;3,6]$, compute $A @ B$ using the first, second, third, and fourth views with separate custom functions, and verify all four results match.

Q4 Slice $A=[1,2,3;4,5,6;7,8,9;10,11,12]$ top-bottom into two row-blocks A_1, A_2 (2 rows each); compute A_1B and A_2B in Python and stack them, then compare against $A @ B$.

Q6 Write a function `block_matmul_sum(A1, A2, B1, B2)` implementing $AB=[A1\ A2][B1;B2]=A_1B_1+A_2B_2$, and compare it with `np.dot(A, B)`.

Q8 Construct a pair of 4×4 matrices A, B , split both into $2\times 2 = 4$ blocks each, and manually verify any one block (e.g. $C_{1,2}$) using the formula from Figure 12.

Questions Q10 – Q18

Q10 Reproduce $A @ B$ using `numpy.einsum('ij,jk->ik', A, B)`, and compare the result against your custom inner-product and outer-product functions.

Q12 Write a function `random_block_partition(A, row_cuts, col_cuts)` that slices matrix A into sub-blocks according to given cut points, returning a list of blocks.

Q14 Explore: if A and B are both split into $3 \times 3 = 9$ blocks instead of $2 \times 2 = 4$, write out the corresponding general block-multiplication formula and verify it in Python.

Q16 Construct two block-diagonal matrices (with $A_{1,2} = A_{2,1} = 0$), verify that their product's off-diagonal blocks are always zero, and explain why.

Q18 Explore: given matrices A, B of shape $(8,8)$, simulate splitting each into 4 blocks distributed to 4 “compute nodes”, computing separately and aggregating into C ; estimate how much computation this parallel scheme could save compared to a single node.

Q11 Try implementing the second view (summed outer products) with `einsum`: write the equivalent `einsum` expression and explain what it means.

Q13 Use the `time` module to compare block-wise computation (slicing then reassembling) against calling $A @ B$ directly, on large matrices (e.g. 400×400).

Q15 Write a function `visualize_block_partition(A, row_cuts, col_cuts)` that draws, with `matplotlib`, a diagram of matrix A split according to the given cut lines.

Q17 Write a function `compare_all_views(A, B)` that computes $A @ B$ using the first, second, third, fourth, and 2×2 -block views (five methods in total), times each, and plots a bar chart comparing them.

Questions Q19 – Q26

Q19 Write a function `block_matmul_general(A_blocks, B_blocks)` that takes arbitrarily shaped block matrices (stored as 2D lists of sub-blocks) and returns the complete block-multiplication result.

Q21 Explore batch matrix multiplication: construct two groups of matrices with shapes $(\text{batch}, 2, 3)$ and $(\text{batch}, 3, 2)$, compute all batch products at once with NumPy's `@` operator or `np.matmul()`, and compare against looping over each batch individually.

Q23 Referring to Table 1, write a function `summarize_views(A, B)` that prints the intermediate results of all four views (the list of inner products, the list of outer-product matrices, the column-combination result, the row-combination result) as a review aid.

Q25 Implement both “slice then reassemble” and “multiply directly as a whole” in Python for a large matrix product, and compare whether the numerical difference between the two stays within floating-point precision.

Q20 Verify in Python: split the identity matrix I into 2×2 blocks ($I_{1,1}$, $I_{2,2}$ being small identity matrices, $I_{1,2}$, $I_{2,1}$ being zero matrices), and check that the block computation of $I @ B$ still equals B .

Q22 Write a function `block_transpose(A_blocks)` that verifies the transpose rule for block matrices: does the block structure of $(AB)^T$ equal swapping block positions and transposing each sub-block?

Q24 Explore: if A is a 2×2 block lower-triangular matrix ($A_{1,2}$ is zero) and B is likewise block lower-triangular, verify whether AB is still block lower-triangular, and derive why using the formula from Figure 12.

Q26 Write a short summary (comments are fine) explaining the relationship between the four views of matrix multiplication and block matrix multiplication, and give the memory trick you find most intuitive.

10 Prompts to Explore Further (1/2)

1

Explain why the four views of matrix multiplication (inner product, outer product, linear combination of columns, linear combination of rows) are, at their core, all special cases of block matrix multiplication.

2

Explain why block matrix multiplication requires the sub-matrix shapes to match pairwise, and what goes wrong if they don't.

3

Explain why $AB = \begin{bmatrix} A1 \\ A2 \end{bmatrix} \begin{bmatrix} B1 & B2 \end{bmatrix}$ (A split top-bottom, B split left-right) resembles the first view of matrix multiplication.

4

Explain why $AB = \begin{bmatrix} A1 & A2 \end{bmatrix} \begin{bmatrix} B1 \\ B2 \end{bmatrix} = A1B1 + A2B2$ (A split left-right, B split top-bottom) resembles the second view of matrix multiplication.

5

Explain the general block-multiplication formula when A and B are both split into four blocks, and how it resembles the rule for multiplying 2×2 matrices.

10 Prompts to Explore Further (2/2)

6

Explain what `np.block()` does, and what advantages it offers over manually reassembling sub-matrices with slicing.

7

Explain how `numpy.einsum` can describe different views of matrix multiplication with one unified string expression, with a couple of concrete examples.

8

Explain the value of block matrix multiplication in parallel or distributed computing — why is large-scale matrix computation so often split into blocks first?

9

Explain the connection between batch `matmul` in deep learning and the block matrix multiplication covered in this section.

10

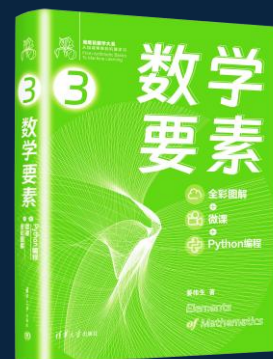
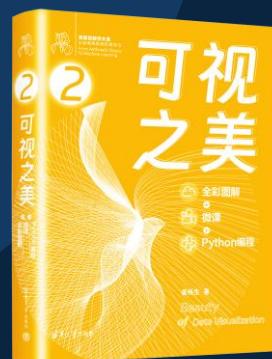
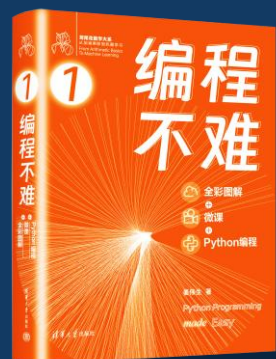
Reflecting on the four views of matrix multiplication and block matrix multiplication together, discuss what it means for learning linear algebra that the same operation can have so many equivalent perspectives.

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

